

Project 1 - Chess Tournament Data

Mohd Afwan Shaikh

2026-09-23

Contents

Approach	1
Introduction	2
Cleaning the Tournament Data	2
Extracting Player Information	3
Validating the Player Data	4
Extracting Opponent Information	5
Calculating Average Opponent Ratings	6
Creating the Final Dataset	7
Exploratory Data Analysis	8
Player Rating vs. Average Opponent Rating	8
Opponent Strength vs. Tournament Performance	10
Distribution of Tournament Points	13
Conclusion	14

Approach

For this project, I will work with a text file containing chess tournament results. The file contains information about each player's name, state, total points, pre-tournament rating, and the opponents they played.

I will use R to clean and organize the text data. I will extract the important information for each player and use the opponent numbers to find each opponent's pre-tournament rating. I will then calculate the average opponent rating for every player.

Finally, I will create a clean data frame containing the player's name, state, total points, pre-rating, and average opponent pre-rating, and export the results as a CSV file.

Introduction

In this project, I will analyze chess tournament results for 64 players using R. My goal is to convert the original text file into a structured dataset containing player information and the average pre-tournament rating of each player's opponents.

I will also check the accuracy of the extracted data and explore the relationship between player ratings, opponent strength, and tournament performance using visualizations.

```
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

library(stringr)

url <- "https://raw.githubusercontent.com/Afwans/DATA607-Project1/main/tournamentinfo.txt"

chess_data <- readLines(url)

## Warning in readLines(url): incomplete final line found on
## 'https://raw.githubusercontent.com/Afwans/DATA607-Project1/main/tournamentinfo.txt'

head(chess_data, 10)
```

```
## [1] "-----"
## [2] " Pair | Player Name          |Total|Round|Round|Round|Round|Round|Round|Round| "
## [3] " Num | USCF ID / Rtg (Pre->Post) | Pts | 1 | 2 | 3 | 4 | 5 | 6 | 7 | "
## [4] "-----"
## [5] "    1 | GARY HUA                |6.0  |W 39|W 21|W 18|W 14|W 7|D 12|D 4| "
## [6] "    ON | 15445895 / R: 1794  ->1817  |N:2  |W   |B   |W   |B   |W   |B   |W   | "
## [7] "-----"
## [8] "    2 | DAKSHESH DARURI              |6.0  |W 63|W 58|L 4|W 17|W 16|W 20|W 7| "
## [9] "    MI | 14598900 / R: 1553  ->1663  |N:2  |B   |W   |B   |W   |B   |W   |B   | "
## [10] "-----"
```

Cleaning the Tournament Data

The original tournament file contains player information, column headers, and separator lines. Each player's information is stored across two lines.

I will first remove the unnecessary headers and separators. This will make it easier to extract the player information and organize it into a structured dataset.

```

# Remove the first four header lines
player_data <- chess_data[-(1:4)]

# Remove separator lines
player_data <- player_data[
  !str_detect(player_data, "\\s*--+\\s*$")
]

# Check the number of remaining lines
length(player_data)

```

```
## [1] 128
```

```

# Display the first six lines
head(player_data, 6)

```

```

## [1] "      1 | GARY HUA                                |6.0 |W 39|W 21|W 18|W 14|W 7|D 12|D 4|"
## [2] "    ON | 15445895 / R: 1794  ->1817             |N:2 |W   |B   |W   |B   |W   |B   |W   |"
## [3] "      2 | DAKSHESH DARURI                        |6.0 |W 63|W 58|L 4|W 17|W 16|W 20|W 7|"
## [4] "    MI | 14598900 / R: 1553  ->1663             |N:2 |B   |W   |B   |W   |B   |W   |B   |"
## [5] "      3 | ADITYA BAJAJ                            |6.0 |L 8|W 61|W 25|W 21|W 11|W 13|W 12|"
## [6] "    MI | 14959604 / R: 1384  ->1640             |N:2 |W   |B   |W   |B   |W   |B   |W   |"

```

Extracting Player Information

After removing the unnecessary lines, I have 128 lines representing 64 players. Each player has two lines of information.

I will separate these lines and extract the player number, name, state, total points, and pre-tournament rating.

```

# Separate the two lines for each player
player_lines <- player_data[seq(1, length(player_data), by = 2)]
rating_lines <- player_data[seq(2, length(player_data), by = 2)]

# Extract player information
players <- data.frame(
  Player_Number = as.integer(
    str_trim(str_extract(player_lines, "\\s*\\d+"))
  ),
  Player_Name = str_trim(
    str_match(player_lines, "\\s*\\d+\\s*\\|([^\|]+)")[, 2]
  ),
  State = str_trim(
    str_match(rating_lines, "\\s*([^\|]+)\\|\\|")[, 2]
  ),
  Total_Points = as.numeric(
    str_trim(str_match(player_lines, "\\s*\\d+\\s*\\|([^\|]+)\\|\\|([^\|]+)")[, 2])
  )
)

```

```

),

Pre_Rating = as.integer(
  str_match(rating_lines, "R:\\s*(\\d+)")[, 2]
)
)

```

```
head(players)
```

```
##   Player_Number      Player_Name State Total_Points Pre_Rating
## 1             1          GARY HUA   ON             6.0      1794
## 2             2    DAKSHESH DARURI   MI             6.0      1553
## 3             3      ADITYA BAJAJ   MI             6.0      1384
## 4             4 PATRICK H SCHILLING   MI             5.5      1716
## 5             5        HANSHI ZUO   MI             5.5      1655
## 6             6        HANSEN SONG   OH             5.0      1686
```

```

# Confirm the number of players
nrow(players)

```

```
## [1] 64
```

```

# Check for missing information
colSums(is.na(players))

```

```
## Player_Number      Player_Name      State Total_Points      Pre_Rating
##             0             0             0             0             0
```

Validating the Player Data

Before calculating the average opponent ratings, I will check that all 64 players were extracted and that none of the required fields contain missing values.

```

# Check the total number of players
nrow(players)

```

```
## [1] 64
```

```

# Check for missing values in each column
colSums(is.na(players))

```

```
## Player_Number      Player_Name      State Total_Points      Pre_Rating
##             0             0             0             0             0
```

```

# Check for duplicate player numbers
sum(duplicated(players$Player_Number))

```

```
## [1] 0
```

```
# Check the range of player numbers
range(players$Player_Number)
```

```
## [1] 1 64
```

Extracting Opponent Information

Each player has seven rounds of tournament results. The results include wins, losses, draws, and sometimes byes or unplayed rounds.

I will extract the opponent numbers from each round. Only rounds containing an actual opponent number will be used to calculate the average opponent rating.

```
# Extract the seven round columns
round_data <- str_split_fixed(player_lines, "\\|", 11)

# Keep only the seven round results
round_data <- round_data[, 4:10]

# Function to extract opponent numbers
get_opponents <- function(rounds) {

  # Extract numbers from rounds with W, L, or D
  opponent_numbers <- str_match(
    rounds,
    "^\\s*[WLD]\\s+(\\d+)\\s*$"
  )[, 2]

  # Remove rounds without opponents
  opponent_numbers <- opponent_numbers[!is.na(opponent_numbers)]

  # Convert opponent numbers to integers
  as.integer(opponent_numbers)
}

# Extract opponents for every player
opponents <- apply(round_data, 1, get_opponents)

# Display Gary Hua's opponents
opponents[[1]]
```

```
## [1] 39 21 18 14 7 12 4
```

```
# Check the number of opponents for each player
table(lengths(opponents))
```

```
##
## 1 3 4 5 6 7
## 1 1 1 7 13 41
```

```
# Check Gary Hua's opponent count
length(opponents[[1]])
```

```
## [1] 7
```

Calculating Average Opponent Ratings

After extracting the opponent numbers, I will match each opponent with their pre-tournament rating.

I will then calculate the average opponent rating for every player. Rounds without an actual opponent will not be included in the calculation.

```
# Function to calculate average opponent rating
get_average_rating <- function(opponent_ids) {

  # Find the pre-ratings of the opponents
  ratings <- players$Pre_Rating[
    match(opponent_ids, players$Player_Number)
  ]

  # Calculate the average
  if (length(ratings) == 0 || any(is.na(ratings))) {
    return(NA_real_)
  }

  round(mean(ratings))
}

# Calculate average opponent ratings for all players
players$Avg_Opponent_Rating <- sapply(
  opponents,
  get_average_rating
)

# Display the first six players
head(players)
```

```
##   Player_Number   Player_Name State Total_Points Pre_Rating
## 1             1      GARY HUA   ON           6.0      1794
## 2             2  DAKSHESH DARURI MI           6.0      1553
## 3             3    ADITYA BAJAJ MI           6.0      1384
## 4             4 PATRICK H SCHILLING MI           5.5      1716
## 5             5     HANSHI ZUO  MI           5.5      1655
## 6             6    HANSEN SONG  OH           5.0      1686
##   Avg_Opponent_Rating
## 1              1605
## 2              1469
## 3              1564
## 4              1574
## 5              1501
## 6              1519
```

```
# Display Gary Hua's opponent ratings
players$Pre_Rating[
  match(opponents[[1]], players$Player_Number)
]
```

```
## [1] 1436 1563 1600 1610 1649 1663 1716
```

```
# Verify his average opponent rating
players$Avg_Opponent_Rating[1]
```

```
## [1] 1605
```

```
# Check for missing averages
sum(is.na(players$Avg_Opponent_Rating))
```

```
## [1] 0
```

Creating the Final Dataset

After extracting and validating the player information, I will create the final dataset containing the five required columns.

I will export this dataset as a CSV file so that it can be used for further analysis.

```
# Create the final dataset
final_data <- players %>%
  select(
    Player_Name,
    State,
    Total_Points,
    Pre_Rating,
    Avg_Opponent_Rating
  )

# Display the first six players
head(final_data)
```

```
##           Player_Name State Total_Points Pre_Rating Avg_Opponent_Rating
## 1           GARY HUA    ON             6.0       1794             1605
## 2      DAKSHESH DARURI    MI             6.0       1553             1469
## 3      ADITYA BAJAJ    MI             6.0       1384             1564
## 4 PATRICK H SCHILLING    MI             5.5       1716             1574
## 5      HANSHI ZUO    MI             5.5       1655             1501
## 6      HANSEN SONG    OH             5.0       1686             1519
```

```
# Check the dimensions
dim(final_data)
```

```
## [1] 64  5
```

```

# Export the dataset
write.csv(
  final_data,
  "chess_tournament_clean.csv",
  row.names = FALSE
)

# Read the exported CSV
check_data <- read.csv("chess_tournament_clean.csv")

# Confirm the dimensions
dim(check_data)

```

```
## [1] 64 5
```

```

# Check for missing values
colSums(is.na(check_data))

```

```

##      Player_Name      State      Total_Points      Pre_Rating
##              0              0              0              0
## Avg_Opponent_Rating
##              0

```

```

# Compare the original and exported datasets
all.equal(final_data, check_data)

```

```
## [1] TRUE
```

```

# Check whether the actual values match
all(
  as.matrix(final_data) == as.matrix(check_data)
)

```

```
## [1] TRUE
```

The exported CSV contains 64 rows and five columns, with no missing values. I compared the exported file with the original data frame to verify that the data was saved correctly.

Exploratory Data Analysis

Now that the dataset has been cleaned and validated, I will explore the relationship between player ratings, opponent strength, and tournament performance.

Player Rating vs. Average Opponent Rating

This visualization compares each player's pre-tournament rating with the average pre-tournament rating of their opponents.

```

library(ggplot2)

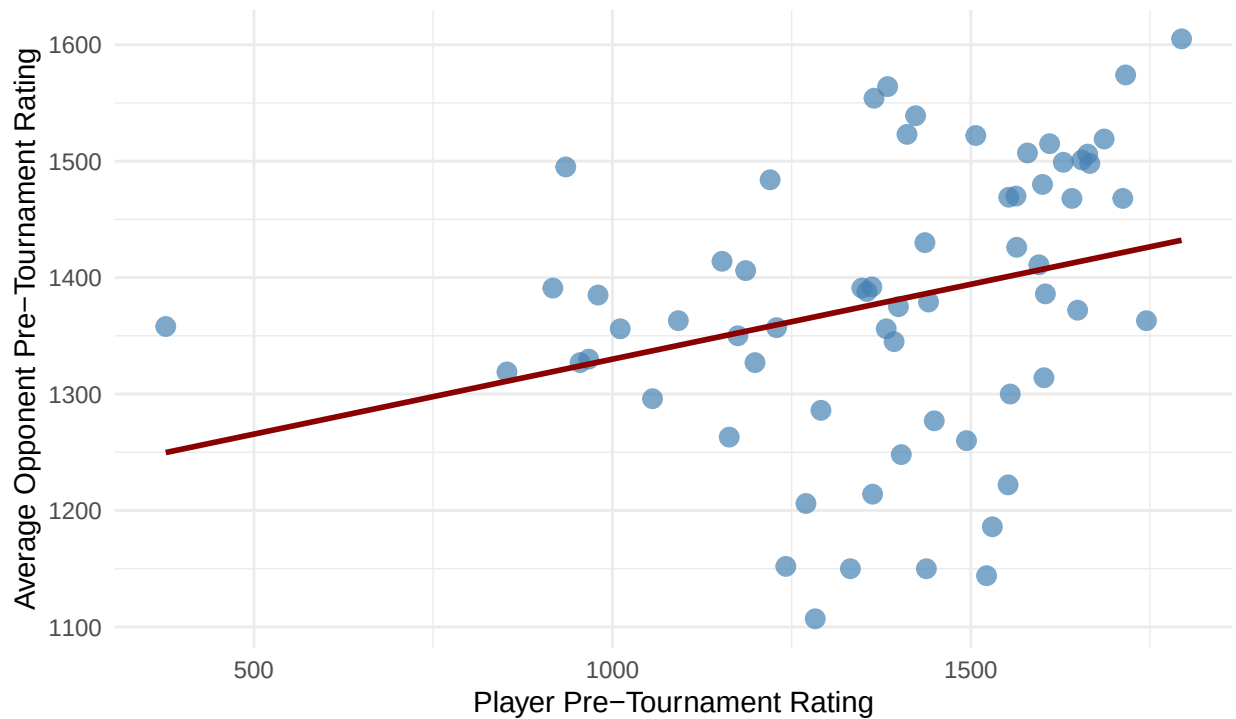
ggplot(final_data, aes(
  x = Pre_Rating,
  y = Avg_Opponent_Rating
)) +
  geom_point(
    color = "steelblue",
    size = 3,
    alpha = 0.7
  ) +
  geom_smooth(
    method = "lm",
    se = FALSE,
    color = "darkred"
  ) +
  labs(
    title = "Player Rating vs. Opponent Strength",
    subtitle = "Chess Tournament Performance Analysis",
    x = "Player Pre-Tournament Rating",
    y = "Average Opponent Pre-Tournament Rating",
    caption = "Source: Chess Tournament Dataset"
  ) +
  theme_minimal()

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

Player Rating vs. Opponent Strength

Chess Tournament Performance Analysis



Source: Chess Tournament Dataset

```
cor(  
  final_data$Pre_Rating,  
  final_data$Avg_Opponent_Rating  
)
```

```
## [1] 0.2839375
```

Interpretation

The correlation between player pre-tournament ratings and average opponent ratings was approximately 0.284.

This indicates a weak positive relationship. Higher-rated players tended to face slightly stronger opponents, but there was considerable variation.

The scatter plot also shows that players with similar pre-tournament ratings sometimes faced opponents with very different average ratings.

This suggests that a player's initial rating alone does not fully explain the strength of their opponents in this tournament.

Opponent Strength vs. Tournament Performance

To further explore tournament performance, I will calculate the difference between each player's pre-tournament rating and the average rating of their opponents.

I will then examine the relationship between this rating difference and total tournament points.

```
# Calculate the rating difference
analysis_data <- final_data %>%
  mutate(
    Rating_Difference = Avg_Opponent_Rating - Pre_Rating
  )

# Display the first six players
head(analysis_data)
```

```
##      Player_Name State Total_Points Pre_Rating Avg_Opponent_Rating
## 1      GARY HUA    ON           6.0      1794           1605
## 2    DAKSHESH DARURI MI           6.0      1553           1469
## 3      ADITYA BAJAJ MI           6.0      1384           1564
## 4 PATRICK H SCHILLING MI           5.5      1716           1574
## 5      HANSHI ZUO  MI           5.5      1655           1501
## 6    HANSEN SONG   OH           5.0      1686           1519
##      Rating_Difference
## 1              -189
## 2              -84
## 3              180
## 4             -142
## 5             -154
## 6             -167
```

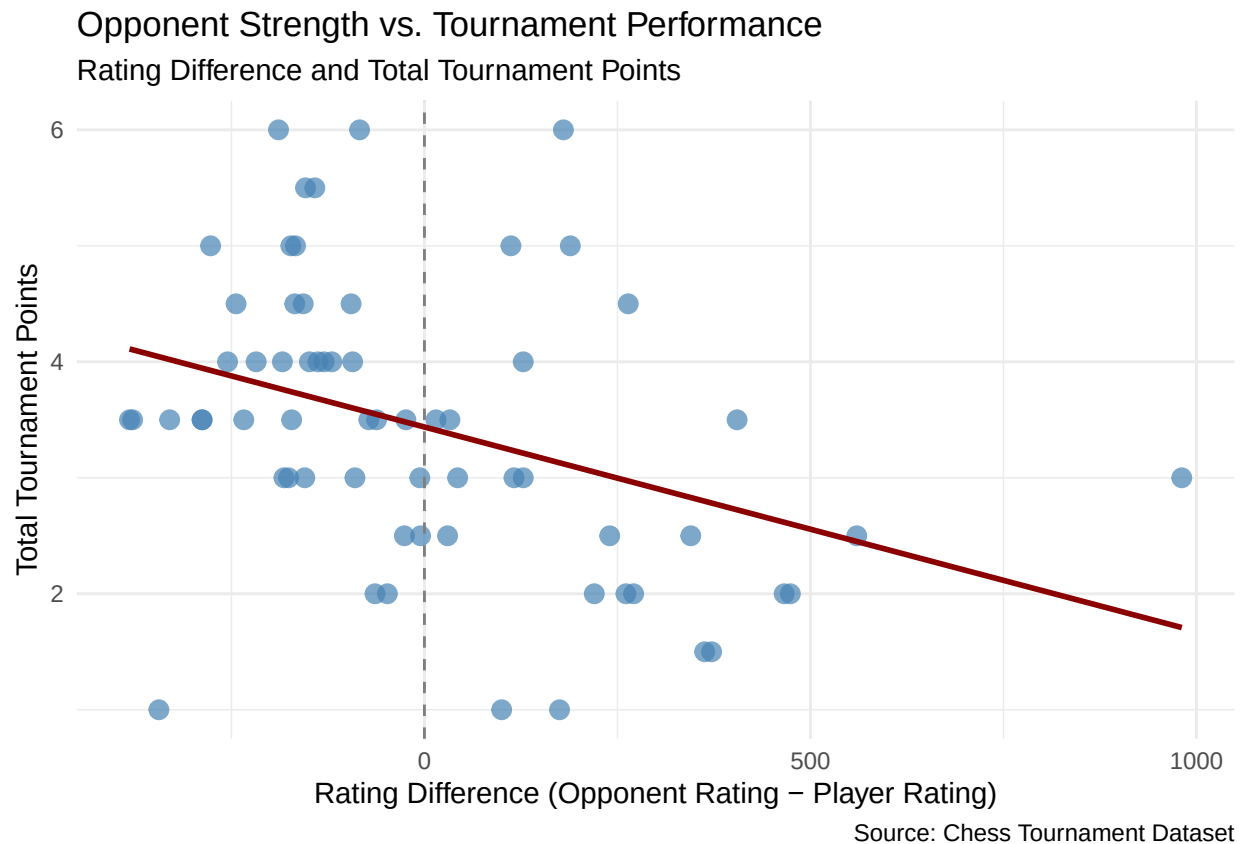
Rating Difference vs. Tournament Points

This visualization explores the relationship between opponent strength relative to a player's rating and the total points earned during the tournament.

```
ggplot(analysis_data, aes(
  x = Rating_Difference,
  y = Total_Points
)) +
  geom_point(
    color = "steelblue",
    size = 3,
    alpha = 0.7
  ) +
  geom_smooth(
    method = "lm",
    se = FALSE,
    color = "darkred"
  ) +
  geom_vline(
    xintercept = 0,
    linetype = "dashed",
    color = "gray50"
  ) +
  labs(
    title = "Opponent Strength vs. Tournament Performance",
    subtitle = "Rating Difference and Total Tournament Points",
  )
```

```
x = "Rating Difference (Opponent Rating - Player Rating)",
y = "Total Tournament Points",
caption = "Source: Chess Tournament Dataset"
) +
theme_minimal()
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



```
# Calculate the correlation
cor(
  analysis_data$Rating_Difference,
  analysis_data$Total_Points
)
```

```
## [1] -0.369523
```

Interpretation

The correlation between rating difference and total tournament points was approximately -0.37, indicating a moderate negative relationship.

In the graph, the red line represents the linear regression trend. Since the line slopes downward, it shows that players who faced stronger opponents relative to their own ratings generally earned fewer tournament points.

However, the scatter plot shows considerable variation. Some players earned high scores despite facing stronger opponents, while others earned fewer points against comparatively weaker opponents.

These findings describe an association within this tournament and do not establish causation.

Distribution of Tournament Points

This visualization shows how tournament points are distributed among all 64 players. It helps identify the most common scores and understand the overall distribution of tournament results.

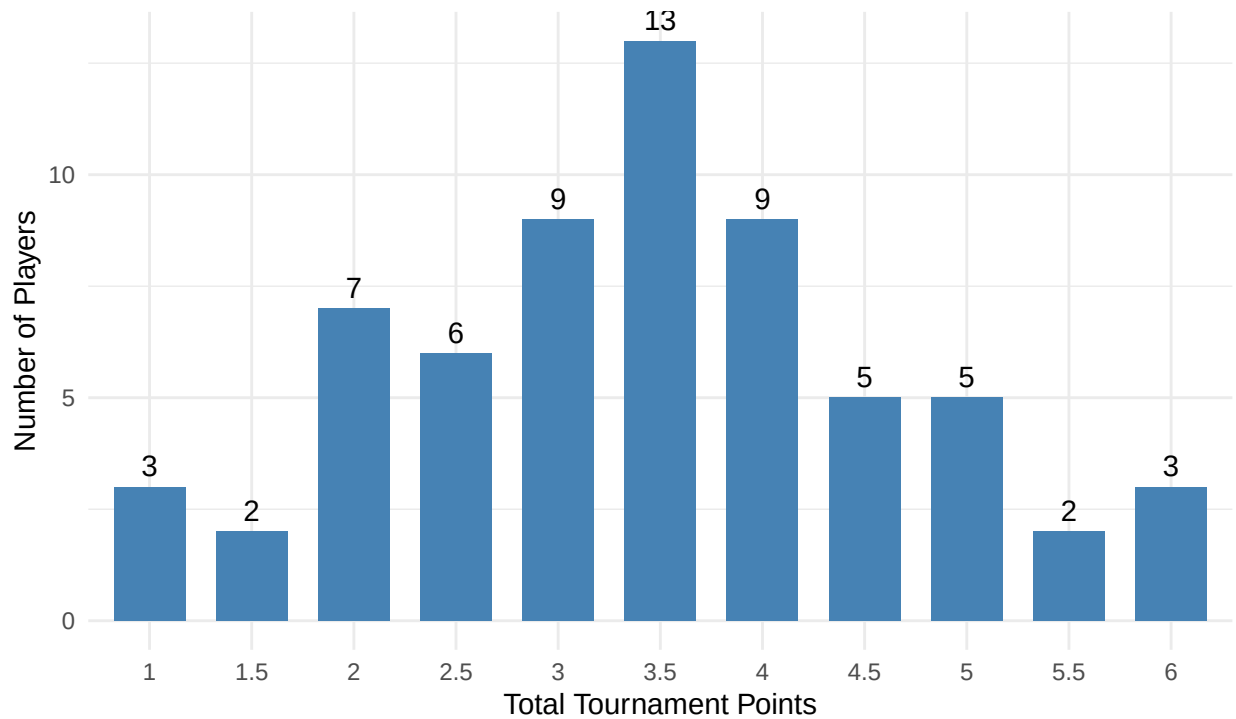
```
# Count the number of players for each score
points_data <- final_data %>%
  count(Total_Points)

# Display the results
points_data
```

##	Total_Points	n
## 1	1.0	3
## 2	1.5	2
## 3	2.0	7
## 4	2.5	6
## 5	3.0	9
## 6	3.5	13
## 7	4.0	9
## 8	4.5	5
## 9	5.0	5
## 10	5.5	2
## 11	6.0	3

```
# Create a bar chart
ggplot(points_data, aes(
  x = factor(Total_Points),
  y = n
)) +
  geom_col(
    fill = "steelblue",
    width = 0.7
  ) +
  geom_text(
    aes(label = n),
    vjust = -0.5,
    size = 4
  ) +
  labs(
    title = "Distribution of Tournament Points",
    subtitle = "Scores Earned by 64 Chess Players",
    x = "Total Tournament Points",
    y = "Number of Players",
    caption = "Source: Chess Tournament Dataset"
  ) +
  theme_minimal()
```

Distribution of Tournament Points
Scores Earned by 64 Chess Players



Source: Chess Tournament Dataset

Interpretation

The bar chart shows that 3.5 points was the most common tournament score, earned by 13 players.

Most players earned between 2 and 4 points. Only three players earned 6 points, while three players earned just 1 point.

The results show that tournament scores were concentrated around the middle, with fewer players achieving the highest or lowest scores.

Conclusion

In this project, I used R to clean and analyze chess tournament data for 64 players. I extracted player information from an unstructured text file and calculated the average pre-tournament rating of each player's opponents.

I validated the dataset by checking for missing values, duplicate player numbers, and calculation accuracy. I then exported the results into a CSV file containing the five required columns.

My analysis revealed three findings:

1. Player ratings and average opponent ratings had a weak positive correlation of 0.284.
2. Rating difference and tournament points had a moderate negative correlation of -0.370.
3. The most common tournament score was 3.5 points, earned by 13 players.

These findings demonstrate how R can transform unstructured text into useful information and support exploratory data analysis.

However, these relationships describe only this tournament and should not be interpreted as evidence of causation.